Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/overview
Exported: 2026-10-14. Source metadata date: 2026-01-26T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

.NET dependency injection

.NET supports the *dependency injection* (DI) software design pattern, which is a technique for achieving Inversion of Control (IoC) between classes and their dependencies. Dependency injection in .NET is a built-in part of the framework, along with configuration, logging, and the options pattern.

> Important
>
> The examples in this article use the `Microsoft.NET.Sdk.Worker` SDK. For more information, see Worker services in .NET.

A *dependency* is an object that another object depends on. The following `MessageWriter` class has a `Write` method that other classes might depend on:

```
public class MessageWriter : IMessageWriter
{
    public void Write(string message)
    {
        Console.WriteLine($"MessageWriter.Write(message: \"{message}\")");
    }
}
```

A class can create an instance of the `MessageWriter` class to use its `Write` method. In the following example, the `MessageWriter` class is a *dependency* of the `Worker` class:

```
public class Worker : BackgroundService
{
    private readonly MessageWriter _messageWriter = new();

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            _messageWriter.Write($"Worker running at: {DateTimeOffset.Now}");
            await Task.Delay(1_000, stoppingToken);
        }
```

```
    }
}
```

In this case, the `Worker` class creates and directly depends on the `MessageWriter` class. Hard-coded dependencies like this are problematic and should be avoided for the following reasons:

- To replace `MessageWriter` with a different implementation, you must modify the `Worker` class.
- If `MessageWriter` has dependencies, the `Worker` class must also configure them. In a large project with multiple classes depending on `MessageWriter`, the configuration code becomes scattered across the app.
- This implementation is difficult to unit test. The app should use a mock or stub `MessageWriter` class, which isn't possible with this approach.

The concept

Dependency injection addresses hard-coded dependency problems through:

- The use of an interface or base class to abstract the dependency implementation.
- Registration of the dependency in a *service container*.

  .NET provides a built-in service container, IServiceProvider. Services are typically registered at the app's start-up and appended to an IServiceCollection. Once all services are added, use BuildServiceProvider to create the service container.

  > Important
  >
  > Desktop apps control their own lifetime. WPF and Windows Forms apps must integrate the host lifetime with the application's lifetime events. For more information, see Use the .NET Generic Host in a WPF app and Use the .NET Generic Host in a Windows Forms app.

- Injection of the service into the constructor of the class where it's used.

  The framework takes on the responsibility of creating an instance of the dependency and disposing of it when it's no longer needed.

> Tip
>
> In dependency injection terminology, a *service* is typically an object that provides a service to other objects, such as the `IMessageWriter` service. The service isn't related to a web service, although it might use a web service.

As an example, assume the `IMessageWriter` interface defines the `Write` method. This interface is implemented by a concrete type, `MessageWriter`, shown previously. The following sample code registers the `IMessageWriter` service with the concrete type `MessageWriter`. The AddSingleton method registers the service with a *singleton* lifetime, which means it isn't disposed until the app shuts down.

Important

The Microsoft.Extensions.Hosting NuGet package provides the types used in this article.

```
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.DependencyInjection;

HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

builder.Services.AddHostedService<Worker>();
builder.Services.AddSingleton<IMessageWriter, MessageWriter>();

using IHost host = builder.Build();

host.Run();

public class MessageWriter : IMessageWriter
{
    public void Write(string message)
    {
        Console.WriteLine($"MessageWriter.Write(message: \"{message}\")");
    }
}

public interface IMessageWriter
{
    void Write(string message);
}

public sealed class Worker(IMessageWriter messageWriter) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            messageWriter.Write($"Worker running at: {DateTimeOffset.Now}");
            await Task.Delay(1_000, stoppingToken);
        }
    }
}
```

In the preceding code example, the highlighted lines:

- Create a host app builder instance.
- Configure the services by registering the `Worker` as a hosted service and the `IMessageWriter` interface as a singleton service with a corresponding implementation of the `MessageWriter` class.
- Build the host and run it.

The host contains the dependency injection service provider. It also contains all the other relevant services required to automatically instantiate the `Worker` and provide the corresponding `IMessageWriter` implementation as an argument.

By using the DI pattern, the worker service doesn't use the concrete type `MessageWriter`, only the `IMessageWriter` interface that it implements. This design makes it easy to change the implementation that the worker service uses without modifying the worker service. The worker service also doesn't *create an*

instance of `MessageWriter`. The DI container creates the instance.

Now, imagine you want to switch out `MessageWriter` with a type that uses the framework-provided logging service. Create a class `LoggingMessageWriter` that depends on ILogger<TCategoryName> by requesting it in the constructor.

```
public class LoggingMessageWriter(
    ILogger<LoggingMessageWriter> logger) : IMessageWriter
{
    public void Write(string message) =>
        logger.LogInformation("Info: {Msg}", message);
}
```

To switch from `MessageWriter` to `LoggingMessageWriter`, simply update the call to `AddSingleton` to register this new `IMessageWriter` implementation:

```
builder.Services.AddSingleton<IMessageWriter, LoggingMessageWriter>();
```

> Tip
>
> The container resolves `ILogger<TCategoryName>` by taking advantage of (generic) open types, which eliminates the need to register every (generic) constructed type.

Constructor injection behavior

Services can be resolved using IServiceProvider (the built-in service container) or ActivatorUtilities. `ActivatorUtilities` creates objects that aren't registered in the container and is used with some framework features.

Constructors can accept arguments that aren't provided by dependency injection, but the arguments must assign default values.

When `IServiceProvider` or `ActivatorUtilities` resolve services, constructor injection requires a *public* constructor.

When `ActivatorUtilities` resolves services, constructor injection requires that only one applicable constructor exists. Constructor overloads are supported, but only one overload can exist whose arguments can all be fulfilled by dependency injection.

Constructor selection rules

When a type defines more than one constructor, the service provider has logic for determining which constructor to use. The constructor with the most parameters where the types are DI-resolvable is selected. Consider the following example service:

```
public class ExampleService
{
    public ExampleService()
```

```
    {
    }

    public ExampleService(ILogger<ExampleService> logger)
    {
        // ...
    }

    public ExampleService(ServiceA serviceA, ServiceB serviceB)
    {
        // ...
    }
}
```

In the preceding code, assume that logging has been added and is resolvable from the service provider but the `ServiceA` and `ServiceB` types aren't. The constructor with the `ILogger<ExampleService>` parameter resolves the `ExampleService` instance. Even though there's a constructor that defines more parameters, the `ServiceA` and `ServiceB` types aren't DI-resolvable.

If there's ambiguity when discovering constructors, an exception is thrown. Consider the following C# example service:

> Warning
>
> This `ExampleService` code with ambiguous DI-resolvable type parameters throws an exception. **Don't** do this—it's intended to show what is meant by "ambiguous DI-resolvable types".

```
public class ExampleService
{
    public ExampleService()
    {
    }

    public ExampleService(ILogger<ExampleService> logger)
    {
        // ...
    }

    public ExampleService(IOptions<ExampleOptions> options)
    {
        // ...
    }
}
```

In the preceding example, there are three constructors. The first constructor is parameterless and requires no services from the service provider. Assume that both logging and options have been added to the DI container and are DI-resolvable services. When the DI container attempts to resolve the `ExampleService` type, it throws an exception, as the two constructors are ambiguous.

Avoid ambiguity by defining a constructor that accepts both DI-resolvable types instead:

```
public class ExampleService
{
    public ExampleService()
```

```
    {
    }

    public ExampleService(
        ILogger<ExampleService> logger,
        IOptions<ExampleOptions> options)
    {
        // ...
    }
}
```

Scope validation

Scoped services are disposed by the container that created them. If a scoped service is created in the root container, the service's lifetime is effectively promoted to singleton because it's only disposed by the root container when the app shuts down. Validating service scopes catches these situations when `BuildServiceProvider` is called.

When an app runs in the development environment and calls CreateApplicationBuilder to build the host, the default service provider performs checks to verify that:

- Scoped services aren't resolved from the root service provider.
- Scoped services aren't injected into singletons.

Scope scenarios

The IServiceScopeFactory is always registered as a singleton, but the IServiceProvider can vary based on the lifetime of the containing class. For example, if you resolve services from a scope, and any of those services take an IServiceProvider, it's a scoped instance.

To achieve scoping services within implementations of IHostedService, such as the BackgroundService, *don't* inject the service dependencies via constructor injection. Instead, inject IServiceScopeFactory, create a scope, then resolve dependencies from the scope to use the appropriate service lifetime.

```
namespace WorkerScope.Example;

public sealed class Worker(
    ILogger<Worker> logger,
    IServiceScopeFactory serviceScopeFactory)
    : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using (IServiceScope scope = serviceScopeFactory.CreateScope())
            {
                try
                {
                    logger.LogInformation(
                        "Starting scoped work, provider hash: {hash}.",
                        scope.ServiceProvider.GetHashCode());

                    var store = scope.ServiceProvider.GetRequiredService<IObjectStore>();
                    var next = await store.GetNextAsync();
```

```
                    logger.LogInformation("{next}", next);

                    var processor = scope.ServiceProvider.GetRequiredService<IObjectProcessor>();
                    await processor.ProcessAsync(next);
                    logger.LogInformation("Processing {name}.", next.Name);

                    var relay = scope.ServiceProvider.GetRequiredService<IObjectRelay>();
                    await relay.RelayAsync(next);
                    logger.LogInformation("Processed results have been relayed.");

                    var marked = await store.MarkAsync(next);
                    logger.LogInformation("Marked as processed: {next}", marked);
                }
                finally
                {
                    logger.LogInformation(
                        "Finished scoped work, provider hash: {hash}.{nl}",
                        scope.ServiceProvider.GetHashCode(), Environment.NewLine);
                }
            }
        }
    }
}
```

In the preceding code, while the app is running, the background service:

- Depends on the IServiceScopeFactory.
- Creates an IServiceScope for resolving other services.
- Resolves scoped services for consumption.
- Works on processing objects and then relaying them, and finally marks them as processed.

From the sample source code, you can see how implementations of IHostedService can benefit from scoped service lifetimes.

Keyed services

You can register services and perform lookups based on a key. In other words, it's possible to register multiple services with different keys and use this key for the lookup.

For example, consider the case where you have different implementations of the interface `IMessageWriter`: `MemoryMessageWriter` and `QueueMessageWriter`.

You can register these services using the overload of the service registration methods (seen earlier) that supports a key as a parameter:

```
services.AddKeyedSingleton<IMessageWriter, MemoryMessageWriter>("memory");
services.AddKeyedSingleton<IMessageWriter, QueueMessageWriter>("queue");
```

The `key` isn't limited to `string`. The `key` can be any `object` you want, as long as the type correctly implements `Equals`.

In the constructor of the class that uses `IMessageWriter`, you add the FromKeyedServicesAttribute to specify the key of the service to resolve:

```
public class ExampleService
{
    public ExampleService(
        [FromKeyedServices("queue")] IMessageWriter writer)
    {
        // Omitted for brevity...
    }
}
```

KeyedService.AnyKey property

The KeyedService.AnyKey property provides a special key for working with keyed services.

Service registration

You can register a service using `KeyedService.AnyKey` as a fallback that matches any key. This is useful when you want to provide a default implementation for any key that doesn't have an explicit registration.

```
var services = new ServiceCollection();

// Register a fallback cache for any key.
services.AddKeyedSingleton<ICache>(KeyedService.AnyKey, (sp, key) =>
{
    // Create a cache instance based on the key.
    return new DefaultCache(key?.ToString() ?? "unknown");
});

// Register a specific cache for the "premium" key.
services.AddKeyedSingleton<ICache>("premium", new PremiumCache());

ServiceProvider provider = services.BuildServiceProvider();

// Requesting with "premium" key returns PremiumCache.
ICache? premiumCache = provider.GetKeyedService<ICache>("premium");
Console.WriteLine($"Premium key: {premiumCache}");

// Requesting with any other key uses the AnyKey fallback.
ICache? basicCache = provider.GetKeyedService<ICache>("basic");
Console.WriteLine($"Basic key: {basicCache}");

ICache? standardCache = provider.GetKeyedService<ICache>("standard");
Console.WriteLine($"Standard key: {standardCache}");

/* This example outputs:
 *
 * Premium key: Premium cache
 * Basic key: basic cache
 * Standard key: standard cache
*/
```

In the preceding example:

- Requesting `ICache` with key `"premium"` returns the `PremiumCache` instance.
- Requesting `ICache` with any other key (like `"basic"` or `"standard"`) creates a new `DefaultCache` using the `AnyKey` fallback.

Query for services

You can query for all services that were registered *using a specific key* (that is, not with `KeyedService.AnyKey`), by passing `KeyedService.AnyKey` to GetKeyedServices<T>(IServiceProvider, Object).

```
IEnumerable<ICache>? keyedCaches = provider.GetKeyedServices<ICache>(KeyedService.AnyKey);
foreach (ICache cache in keyedCaches)
{
    Console.WriteLine($"AnyKey registered cache: {cache}");
}

/* This example outputs:
 *
 * AnyKey registered cache: Premium cache
*/
```

In the preceding example, calling `GetKeyedServices<T>(KeyedService.AnyKey)` returns only the `PremiumCache` instance since it's the only cache that was registered using a specific key in the service registration code example.

> Important
>
> Starting in .NET 10, calling `GetKeyedService()` (singular) with `KeyedService.AnyKey` throws an InvalidOperationException because `AnyKey` shouldn't be used to resolve a single service. For more information, see Fix issues in GetKeyedService() and GetKeyedServices() with AnyKey.

See also

- Quickstart: Dependency injection basics
- Tutorial: Use dependency injection in .NET
- Dependency injection guidelines
- Dependency injection in ASP.NET Core
- NDC Conference Patterns for DI app development
- Explicit dependencies principle
- Inversion of control containers and the dependency injection pattern (Martin Fowler)